

inside PIM and sends the results to an NPU for Softmax computation. However, this introduces data transfer overhead proportional to L_{seq} , and although PIM–NPU pipelining reduces the stall time, its use of two row buffers increases power consumption by $1.74\times$. PIM-GPT [30] integrates Taylor-series and Newton–Raphson–based approximation units to support Softmax, but these dedicated units incur an overhead of approximately 0.64mm^2 area and 304.59mW power. AttAcc [2] and Duplex [4] integrate Softmax units into the logic die to reduce memory traffic, but the area costs are substantial: AttAcc’s Softmax module occupies 1.38mm^2 , which is roughly $69\times$ the area of the accumulator logic, and Duplex requires a Softmax unit approximately $17\times$ larger than its associated GEMM module.

Sparsity-exploiting PIM architectures. Leveraging sparsity in matrix computations can reduce both computation and memory traffic by skipping zero or low-importance elements. From this perspective, several sparsity-aware PIM architectures attempt to avoid sending unnecessary data to external computing units and instead process it within or near memory to reduce data movement and latency [31], [27], [32]. SpaceA [31] deploys processing elements per bank and uses a content-addressable memory–based cache to identify and process only non-zero elements, achieving $13.54\times$ speedup over a Titan Xp GPU. pSyncPIM [27] enables per-bank early stopping even under all-bank execution through predicated execution, but its design introduces roughly $13.26\times$ area overhead compared to SpaceA. PIM-Potential [32] extends the HBM-PIM architecture [21] and issues selective instructions under GPU-driven orchestration, achieving up to $3.00\times$ speedup over a GPU. However, all these techniques depend on sparsity patterns known in advance, making them unsuitable for LLM SDPA workloads where activation sparsity varies dynamically. Consequently, they cannot detect or exploit dynamic sparsity inside memory or execute the entire SDPA computation as an in-memory stream.

Overall, prior HW/SW co-design studies reduce the computational load of Softmax through approximation but still retain expensive operations such as the exponential and division, or preserve the global reductions, leaving its sequential bottleneck unresolved. PIM-based SDPA accelerators mitigate the memory-bound nature by moving GEMV into memory, but Softmax remains handled either off-chip or via large dedicated units with substantial area overhead. Sparsity-exploiting PIM architectures are optimized for static sparsity patterns and therefore cannot adapt to the dynamic sparsity exhibited by SDPA activations. These limitations highlight the need for (i) a new function that fundamentally avoids Softmax’s structural constraints, including token dependencies and global reductions, and (ii) an architecture that can immediately exploit the dynamic sparsity produced by such a function inside PIM to skip unnecessary computation and memory accesses.

IV. ELEMEX: A SPARSE AND PARALLEL REPLACEMENT FOR SOFTMAX

In this section, we propose Elemex to address three key limitations of Softmax in the MHA: (i) long latency caused by

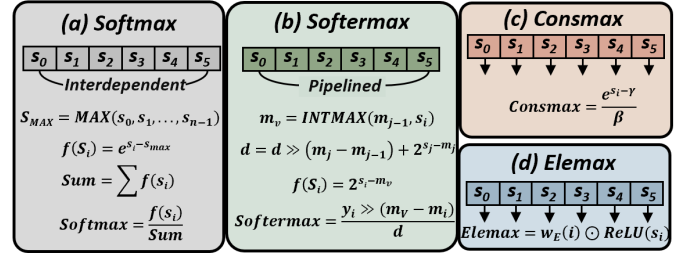


Fig. 3. Computational structures of Softmax, Softmax [16], Consmax [11], and Elemex.

nonlinear and global reduction operations, (ii) accuracy degradation due to assigning non-negligible attention to all tokens, and (iii) limited interpretability because uniform normalization does not reflect the positional importance of tokens within the context [6], [26]. Elemex is designed with minimal operations to relax these structural and semantic constraints, and its form naturally aligns with WTA behavior, enabling opportunities for accelerated execution.

A. Elemex Design

To implement a lightweight computation that preserves the positive output property of Softmax and its WTA behavior, we design Elemex for the i -th score vector $\mathbf{s}_i$ as follows:

$$\text{Elemex}(\mathbf{s}_i) = \mathbf{w}_E(i) \odot \text{ReLU}(\mathbf{s}_i), \quad (2)$$

As shown in Eq. (2), Elemex applies ReLU to zero out negative inputs and thereby reproduces key Softmax characteristics with a much simpler computation path. Its normalization effect is achieved by multiplying a learnable parameter vector $\mathbf{w}_E \in \mathbb{R}^N$ —defined per layer, head, and token—without enforcing a probability distribution. This stabilizes the magnitude and variance of S while removing the need for division-based normalization.

Fig. 3 provides a visual comparison of the computation flows of Softmax [16], Consmax [11], and Elemex as Softmax replacements for generative LLMs. Unlike Softmax, Softmax, and Consmax, Elemex does not use the exponential operation, eliminating both the maximum search and subtraction step and the numerical risks associated with division. Moreover, Elemex processes each element of the input vector independently, enabling high parallelism. Because it does not require synchronization across all elements of S for summation—unlike methods that wait for all exponentials to complete—its latency does not escalate sharply even for long-sequence inputs.

Moreover, Elemex does not constrain the sum of its outputs to 1; instead, it is designed to control sparsity and interpretability in a token-wise manner. In Softmax, the outputs for each query are normalized to sum to 1, so as L_{seq} increases, the average share that each token can receive decreases to approximately $1/L_{\text{seq}}$. When the number of actually important tokens also grows with the sequence length, this limited budget must be distributed across more relevant tokens, which reduces

the absolute attention value each important token can obtain. As a result, the attention distribution becomes flat, known as attention dilution [6]. In contrast, Elemax does not impose such a normalization budget and can therefore form attention patterns that are less sensitive to changes in sequence length. In addition, because ReLU immediately sets negative values to 0, Elemax naturally excludes low-importance tokens and preserves contextual sparsity, in line with prior work that explicitly enforces sparse attention distributions [25], [24]. Finally, the token-wise learnable parameter w_E encodes the contextual importance of each position and allows the model to independently adjust the required focus strength at each token, thereby improving interpretability.

B. Training Objective

Applying ReLU-based attention directly to self-attention leads to two issues. First, unlike Softmax, ReLU does not normalize the scores to sum to 1, which causes the variance of the self-attention output to grow proportionally with the sequence length L_{seq} . This variance explosion results in unstable training. Second, because ReLU maps all negative scores to 0, the resulting attention distribution becomes overly sparse. Only a small number of tokens contribute meaningfully to context aggregation, while most tokens are ignored. Therefore, rather than merely increasing or decreasing sparsity, it is essential to construct a ReLU-based attention distribution that stabilizes the variance with respect to L_{seq} while avoiding excessive sparsification. To achieve this goal, we introduce a loss term that guides w_E to adjust the scale of the ReLU-based distribution according to the sequence length and to prevent many tokens from collapsing to zero.

Notation. If we denote the scaled dot-product between the i -th query vector $\mathbf{q}_i$ and the j -th key vector $\mathbf{k}_j$ as $s_{ij} = \mathbf{q}_i \mathbf{k}_j^\top$, then the unnormalized probability p_{ij} produced by Elemax and the normalized probability $\hat{p}_{ij}$ used during training are defined as in Eq. (3):

$$p_{ij} = w_E(i) \text{ReLU}(s_{ij}), \quad \hat{p}_{ij} = \frac{p_{ij}}{\sum_{j'} p_{ij'}} \quad (3)$$

Here, $w_E(i)$ denotes a token-wise learnable parameter associated with query position i .

Length-aware scale matching. Let the value vector for each token be denoted by $\mathbf{v}_j$, and assume that its components are mutually independent with variance σ_v^2 . When an arbitrary component of the value vector is denoted by v_j , the variance of the corresponding component in the context vector of token i can be approximated as in Eq. (4):

$$\text{Var}\left[\sum_j p_{ij} v_j\right] \approx w_E(i)^2 \frac{n_{\text{eff}}(i)}{2} \sigma_v^2. \quad (4)$$

Here, $\text{Var}[\cdot]$ denotes the variance operator, and the effective receptive length $n_{\text{eff}}(i)$ represents the number of valid tokens actually referenced by query i . In this expression, the variance of the context vector components for token i is proportional to $n_{\text{eff}}(i)$, so the variance increases linearly as $n_{\text{eff}}(i)$ grows. This implies that even when v_j shares the same distribution across tokens, the scale of the attention output systematically changes with the sequence length, and for longer sequences the norm of

the context vector becomes excessively large, which degrades training stability. To control this variance growth, we introduce an additional loss term $\mathcal{L}_{\text{scale}}$ for Elemax and encourage $w_E(i)$ to learn the scaling pattern $w_E(i) \propto 1/\sqrt{n_{\text{eff}}(i)}$ as a function of the number of effective tokens. To this end, we define the target value $w_E^*(i)$ of the Elemax parameter for token i as follows:

$$w_E^*(i) = \frac{\gamma_h}{\sqrt{n_{\text{eff}}(i)/2}} \quad (5)$$

Here, h denotes the head index in MHA, and γ_h is a learnable scalar associated with head h . We then introduce the following scale loss $\mathcal{L}_{\text{scale}}$ so that the actual $w_E(i)$ follows Eq. (5):

$$\mathcal{L}_{\text{scale}} = \frac{\lambda_{\text{scale}}}{N} \sum_i \left(\log w_E(i) - \log w_E^*(i) \right)^2, \quad (6)$$

Here, N denotes the total number of tokens in the batch, and λ_{scale} is a hyperparameter that controls the strength of $\mathcal{L}_{\text{scale}}$. As shown in Eq. (6), the scale loss $\mathcal{L}_{\text{scale}}$ directly penalizes the discrepancy between $w_E(i)$ and $w_E^*(i)$, enforcing the relationship $w_E(i) \propto 1/\sqrt{n_{\text{eff}}(i)}$ across positions so that the variance of the context output does not explode with $n_{\text{eff}}(i)$.

Distribution regularization. To address the second issue of excessive sparsification in ReLU-based attention, we adapt the regularization loss proposed in ReLUformer [14] to our formulation of p_{ij} . First, the entropy of $\hat{\mathbf{p}}_i$, denoted by $H(\hat{\mathbf{p}}_i)$, is defined as in Eq. (7):

$$H(\hat{\mathbf{p}}_i) = - \sum_j \hat{p}_{ij} \log(\hat{p}_{ij}) \quad (7)$$

Using this entropy, we define the regularization loss $\mathcal{L}_{\text{reg}}$ as in Eq. (8):

$$\mathcal{L}_{\text{reg}} = \frac{\lambda_{\text{reg}}}{N} \sum_i \log \left(\sum_j p_{ij} \right) + \frac{\lambda_{\text{reg}}}{N} \sum_i \max \left(H(\hat{\mathbf{p}}_i) - C(n_{\text{eff}}(i)), 0 \right). \quad (8)$$

Here, λ_{reg} is a hyperparameter that controls the strength of $\mathcal{L}_{\text{reg}}$, and $C(\cdot)$ denotes the entropy upper bound as a function of length. Following ReLUformer [14], we set $C(\cdot)$ as follows:

$$C(n_{\text{eff}}(i)) = \alpha \log n_{\text{eff}}(i) \quad (9)$$

Here, α is a hyperparameter that controls the magnitude of the entropy upper bound. The first term in Eq. (8) prevents $\sum_j p_{ij}$ from becoming excessively small, thereby avoiding overly large sparsity in P . The second term applies a penalty only when the entropy exceeds $C(n_{\text{eff}}(i))$, suppressing over-uniform distributions and guiding the Elemax attention distribution not to become overly diffuse.

Final loss function. Finally, by combining all the components above, we define the loss function as in Eq. (10):

$$\mathcal{L}_{\text{final}} = \mathcal{L}_{\text{task}} + \mathcal{L}_{\text{scale}} + \mathcal{L}_{\text{reg}}, \quad (10)$$

where $\mathcal{L}_{\text{task}}$ is the standard cross-entropy-based task loss, $\mathcal{L}_{\text{scale}}$ prevents the variance from exploding as the sequence length increases, and $\mathcal{L}_{\text{reg}}$ ensures that the Elemax distribution covers sufficient context without attention dilution. We train the Elemax-based LLM using this loss function $\mathcal{L}_{\text{final}}$.

V. SPARE-PIM: PIM-BASED DYNAMIC-SPARSITY ACCELERATOR

SPARE-PIM architecture is a PIM that executes sparse kernels which skip unnecessary memory accesses and computations, thereby reducing latency and energy consumption. Specifically, *SPARE-PIM* uses the bitmask produced during Elemax to obtain the valid operation count (VOC), and the memory controller leverages this information in the subsequent *PV* stage so that the corresponding operations incur only the minimum required latency. To support this execution model, we assume that all PIM and memory commands in the *SPARE-PIM* architecture are issued in order and that out-of-order command issuing is disabled [21].

A. Hardware architecture

Fig. 4 shows the overall hardware organization of *SPARE-PIM* architecture. First, as illustrated in Fig. 4(a), *SPARE-PIM* architecture is an HBM based 8-stack PIM architecture, where eight DRAM dies and one logic die are stacked in 3D and connected through through-silicon vias (TSVs). Each DRAM die contains four channels, and each channel is divided into two pseudo channels (pCH). A single pCH consists of four bankgroups, and each bankgroup comprises four banks. A bank-level processing unit (BPU) and a bankgroup metadata unit (BGMU) are placed at each bank and bankgroup to exploit dynamic sparsity. To allocate sufficient area for the in-memory PIM logic, we also follow prior work [21], [27] and adopt a configuration that halves the number of subarrays defined in JEDEC HBM3 [33].

Fig. 4(b) shows the BPU placed at each bank in the DRAM die. The BPU consists of several logic blocks to exploit the dynamic sparsity produced by Elemax, and *SPARE-PIM* places one BPU in every bank to fully utilize the internal memory bandwidth at the bank level. The main components inside a BPU are as follows:

- The floating-point SIMD unit (FSU) consists of half-precision floating-point (FP16) multipliers and adders, and performs operations on 16 elements simultaneously in a single-instruction, multiple-data (SIMD) fashion.
- The vector buffer (VB) stores activation data (*i.e.*, Q , S , P), and there are two VBs (*i.e.*, VB-A and VB-B). Since each VB can store two vectors, each consisting of sixteen 16-bit elements, a BPU can process up to 32 activation values in a single flow. VB-A also includes a 16-bit scalar register (SCR), which is used to broadcast one element from VB-A to the arithmetic units inside the FSU. VB-B contains simple gate-level logic that uses the sign bits of the stored data to apply ReLU to all elements in VB-B and then generates a bitmask based on the result. In addition, VB-B is connected to the bank I/O so that it can send and receive data in 256-bit chunks.
- The bitmask buffer stores the bitmask generated when ReLU is executed. It can hold two sets of 16-bit values, and each bit in the bitmask buffer corresponds to the order of the activations stored in VB-B.
- The dynamic sparsity selection engine (DSSE) consists of a valid data index (VDI) extractor, which identifies

the positions of non-zero elements based on the bitmask, and an address calculator, which computes the valid data address (VDA) for accessing the bank and VB. In addition, a VOC counter is included to determine the number of valid operations.

- The local controller contains a config file that stores model configuration parameters required for computation, and includes logic that decodes commands delivered from the memory controller and controls the timing of the FSU, buffers, and DSSE inside the BPU.

Fig. 4(c) shows the internal logic organization and behavior of the BGMU. First, the VOC value delivered from the BPU is stored in a register inside the BGMU. When the memory controller issues a BGMU read request, the command decoder in the BGMU controls an internal switch so that the BGMU register is connected to the bankgroup I/O path, identical to a normal bank access. As a result, the VOC data is returned to the memory controller through the same interface as the read data of a regular READ command, without requiring a dedicated path. To distinguish a BGMU read from a normal bank read, we reuse the most significant bits of the row address. Because our architecture places only half of the subarrays, the MSB of the row address is not used for actual bank accesses and remains as a reserved bit. We use this bit as the BGMU selection bit: when the MSB of the row address in a READ command is 0, the command is interpreted as a normal bank access, and when it is 1, the command is interpreted as a BGMU access.

On the logic die, as shown in Fig. 4(a), a channel-level accumulator (CA) is placed for each pCH. The CA sequentially collects partial-sums generated by multiple BPUs within a pCH and accumulates them using an internal accumulator. This design allows *SPARE-PIM* to fully exploit the bank-level bandwidth it provides while enabling parallel processing.

B. Commands

To enable the operation of *SPARE-PIM* architecture, we introduce five new commands. These commands consist of all-bank (AB) execution commands (*i.e.*, MAC, EMUL, MOV, SPM), which are executed synchronously by all BPUs in a channel, and a single-bank (SB) execution command (*i.e.*, ACC), which is executed only by a designated BPU. The role and timing constraints of each command are summarized in Table I, and their detailed behaviors are as follows:

- MAC selects data from the sense amplifier and the VB based on the address encoded in the command, performs multiply-accumulate (MAC) operations in the FSU, and accumulates the partial-sum results in the VB.
- EMUL performs element-wise multiplication between two vectors. Similar to MAC, it drives the multipliers in the FSU according to the address specified in the command and writes the results back to the VB.
- MOV is used to move data between the sense amplifier and the VB. A flag in the command determines whether ReLU is applied to the data stored in VB-B.
- SPM is a long-data command that performs sparse multiplication while reflecting sparsity. Once issued, it follows

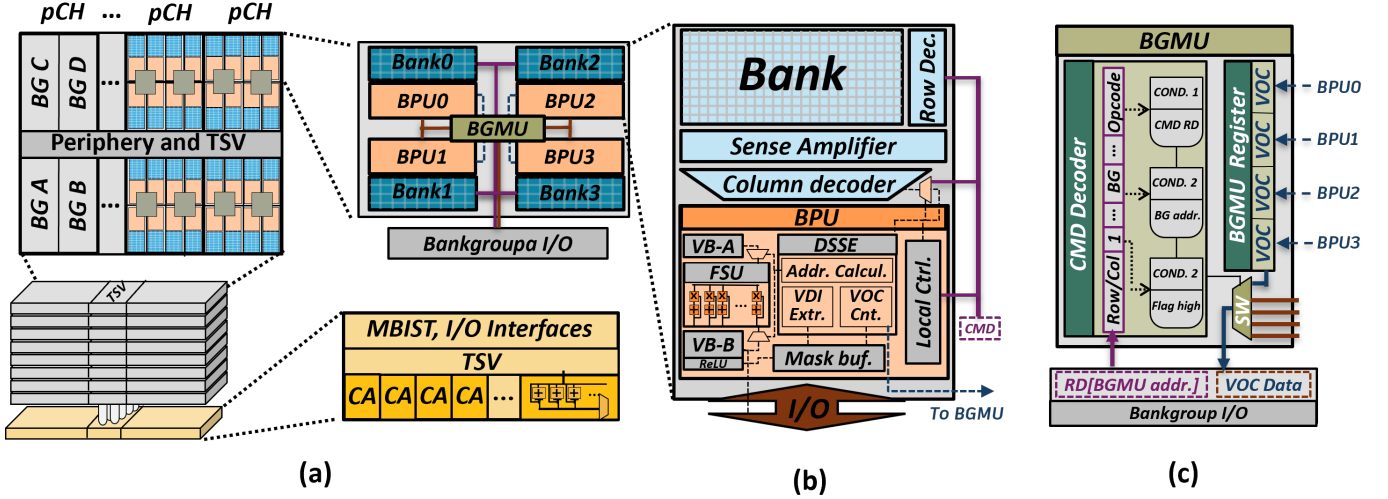


Fig. 4. SPARE-PIM hardware architecture: (a) 8-stack HBM-based PIM with per-bank BPUs and per-bankgroup BGMUs, and per-pCH accumulators, (b) microarchitecture of BPUs, and (c) microarchitecture and operation of BGMUs.

TABLE I
SPARE-PIM COMMANDS

CMD	Description	Timing	Exec.
MAC	Execute MAC.	t_{CCDL}	AB
EMUL	Execute element-wise multiplication.	t_{CCDL}	AB
MOV	Move data between VB and bank (optional ReLU).	t_{CCDS}	AB
SPM	Execute sparse kernel (long-data command).	—	AB
ACC	Accumulate channel-level partial-sum.	$t_{CCDL} + 1$	SB

the address sequence provided by the DSSE to access memory and repeatedly performs MAC operations for VOC times, while the memory controller dynamically sets the execution latency of SPM in proportion to VOC.

- When ACC is issued, the designated bank sends the partial-sum data stored in the VB to the logic die through the I/O path, and the CA accumulates the incoming values for each issued command to produce the final result.

All SPARE-PIM command set is mapped to the reserved-for-future-use (RFU) opcodes in the JEDEC command space while preserving the standard HBM3 interface. All SPARE-PIM architecture commands share the same command/address bus as the standard ACT, READ, and WR commands, and the memory controller schedules them under the same timing constraints as regular HBM commands.

C. Execution flow

By replacing Softmax with Elemax, we propose ESEF, which computes SDPA in a single pass by exploiting the simplified computation flow and eliminating the overhead of storing intermediate activations in the banks. First, as shown in Fig. 5(a), SPARE-PIM architecture repeatedly issues MAC commands that use Q stored in VB-A and K^T data stored in the banks to perform MAC operations. Each result is accumulated in VB-B, and this process continues until the final S

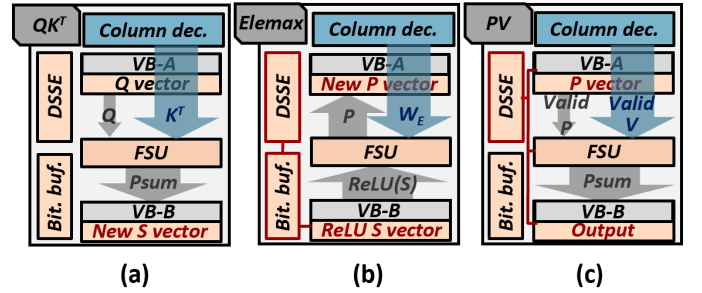


Fig. 5. Elemax-based SDPA execution flow (ESEF): (a) computation of QK^T to form the score vector S , (b) Elemax activation producing P and bitmasks, and (c) sparse PV multiplication.

vector is formed. Once S is completed, the memory controller issues a MOV command and applies ReLU to S stored in VB-B, as illustrated in Fig. 5(b). During this step, the ReLU logic simultaneously generates the corresponding bitmask, and the generated bitmask is stored in the bitmask buffer. The DSSE then uses this bitmask information to prepare for the subsequent PV computation. After ReLU, when w_E is ready to be loaded from the bank into the computation units, an EMUL command performs element-wise multiplication, and the resulting P is stored in VB-A. Finally, for the PV computation, the memory controller issues an SPM command, as shown in Fig. 5(c). In this step, the BPU selectively accesses only valid data based on the addresses generated by the DSSE and performs sparse PV operations. Overall, ESEF computes SDPA in a single pass while exploiting activation sparsity to reduce the latency of PV .

D. Data Mapping

ESEF achieves its latency benefit only when the K^T/V data is placed in the DRAM banks according to the access pattern required by the computation units. To this end, we propose ESDM, a K^T/V data mapping scheme that follows

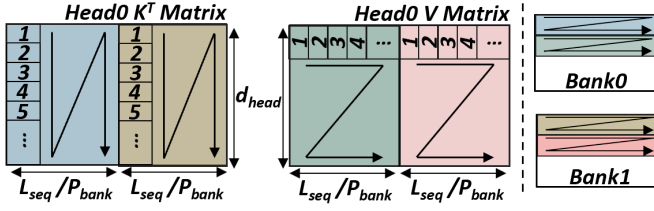


Fig. 6. Elemax-based SDPA data mapping (ESDM) for a single attention head, showing K^T/V layout across banks.

ESEF while maximizing bank-level parallelism through all-bank execution. Let P_{bank} denote the target degree of bank parallelism for a single head. Fig. 6 illustrates the K^T/V layout for the case of $P_{\text{bank}} = 2$. We define an element chunk as a group of consecutive elements processed simultaneously by the FSU in SIMD fashion.

In ESDM, the K^T/V elements associated with a single Q or P element are placed so that the FSU can process them simultaneously during MAC. Specifically, the K^T matrix is arranged so that consecutive element chunks along the d_{head} dimension reside in adjacent DRAM columns within the same row, and the entire d_{head} dimension is stored inside each bank. This ensures that the generation of S from QK is completed locally within a bank. Meanwhile, the V matrix is arranged so that consecutive element chunks along L_{seq} appear in adjacent DRAM columns of a row. This placement allows accessing only the V elements corresponding to the activated P elements determined by Elemax during the PV stage, thereby reducing unnecessary memory accesses. As a result, the FSU can read the required K^T/V data in a continuous burst under page-hit conditions when processing a single Q or P element, minimizing the number of ACT and PRE commands required to repeatedly open and close DRAM rows.

When SDPA is executed with ESDM, each BPU computes only the dot-product for a token range of size $L_{\text{seq}}/P_{\text{bank}}$ during the final PV stage of ESEF. Thus, when $P_{\text{bank}} \geq 2$, each bank produces only partial sums. The CA placed in each channel collects and accumulates these partial sums through the ACC command to form the final SDPA output. In addition, the Elemax parameters are placed in the same bank and aligned with the indices of S stored in VB-B, allowing element-wise multiplication to occur immediately without additional reordering. When the batch size increases, different batches are first distributed across different channels whenever possible. If further channel-level distribution is not possible, additional batches are stored sequentially in the remaining row space of each bank while preserving ESDM. This maintains a high page-hit rate and maximizes the internal bandwidth utilization of SPARE-PIM architecture.

E. Deploying dynamic sparsity

For SPARE-PIM architecture to accelerate sparse PV , the DSSE placed in each BPU must rapidly generate metadata such as VDI and VOC and use it to ensure that only valid data is accessed and processed. To achieve this, we overlap the metadata generation with the Elemax execution time and

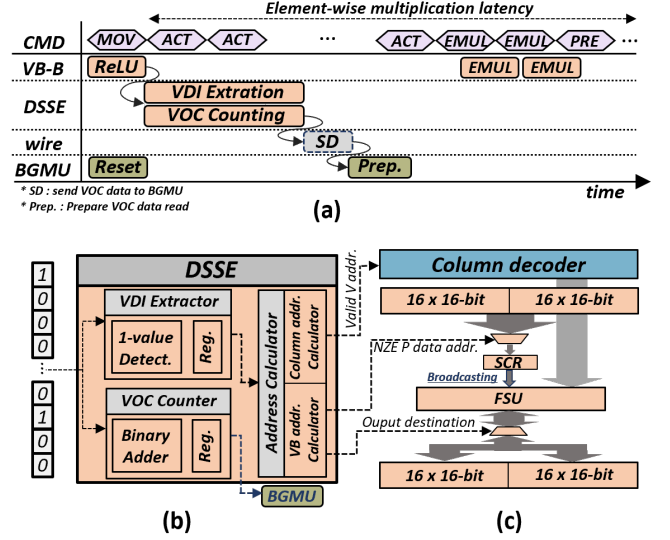


Fig. 7. Deploying dynamic sparsity inside the BPU: (a) timing of metadata generation from Elemax, (b) architecture of the DSSE, and (c) micro-dataflow in FSU when the sparse kernel is executed.

design the DSSE and FSU to perform memory accesses and computations only on valid data.

Fig. 7(a) presents a timing diagram showing how the latency of metadata generation is hidden during Elemax. When a MOV command triggers ReLU, the generated bitmask is immediately used by the VDI extractor to obtain the VDI, which is stored in a register, while the VOC counter computes the VOC in parallel. At the MOV execution point, the BGMU resets its register in preparation for the new VOC. Once VOC generation completes, the new value is transferred to the BGMU in the next cycle. The stored VOC is later used in the BGMU read stage to quickly determine the valid operation length for the channel. Because this logic is independent of Elemax computation, the process of generating and storing metadata can fully overlap with the element-wise multiplication of Elemax without additional latency.

Fig. 7(b) and Fig. 7(c) illustrate how the DSSE generates metadata from the bitmask and how this metadata guides the sparse PV computation in the FSU. When SPM is issued, the DSSE sequentially reads the stored VDI elements and supplies them to the address calculator. The VB address calculator converts each entry into VB-A/B addresses so that VB-A selectively reads only the non-zero P elements, while VB-B receives the destinations where MAC results will be written. In parallel, the column address calculator computes the column address of the V element corresponding to each non-zero P entry and sends it to the column decoder, which selects and forwards the valid V data to the FSU. The FSU then performs MAC operations on the selected P and V values and stores the result into the designated slot within the two 16×16 -bit arrays located beneath the FSU. During the execution of the sparse kernel, VDI-based address generation, selective P/V retrieval, MAC computation, and result storage operate in a pipelined manner. Accessing only the data required by sparsity reduces both memory-access latency and energy consumption.

Algorithm 1 Flow for issuing SPM commands in the memory controller

Require: BGMU aggregates VOCs from BPUs.

```

1: /* Read VOCs from BGMUs and find max_count */
2: max_count ← 0;
3: for bg = 0 to  $N_{BG} - 1$  do
4:   tmp ← READ(BGMR[bg]);
5:   if tmp > max_count then
6:     max_count ← tmp;
7:   end if
8: end for
9: /* Issue SPM commands */
10: if max_count = 0 then
11:   skip {ACT, SPM, PRE};
12: else
13:   for each bank in the channel do
14:     issue ACT;
15:   end for
16:   issue SPM(max_count);
17:   for each bank in the channel do
18:     issue PRE;
19:   end for
20: end if
21: return ;

```

F. Sparse Kernel Execution

For operation skipping in *SPARE-PIM* to have a real impact on inference latency, the architecture must ensure that the sparse kernel executes only for the minimum latency proportional to VOC. To achieve this, the memory controller reads the VOC values from the BGMUs placed in each bankgroup, determines the maximum VOC among them, and configures the latency of the SPM command accordingly before issuing it. The control flow for determining the SPM execution latency is summarized in Algorithm 1.

As shown in Fig. 4(c), after Elemax computation completes, the memory controller issues a standard READ command with the BG address and a flag (*i.e.*, MSB of the row address) set high. This READ command is delivered to the BGMU's command decoder, and when it is identified as a BGMU-targeted access, the BGMU connects the VOC data stored in its register to the bankgroup I/O path and returns it to the controller. The BGMU read latency follows the standard HBM3 timing constraints and is modeled as Eq. (11):

$$L_{BGMU} = t_{CL} + (N_{BG} - 1)t_{CCD_S} + t_{BURST}. \quad (11)$$

Here, N_{BG} denotes the number of bankgroups in a channel. The first term t_{CL} is the column access latency of the first READ, $(N_{BG} - 1)t_{CCD_S}$ captures the spacing between consecutive READ commands, and the last term t_{BURST} is the final data-burst duration. The BGMU read latency is the sum of these components.

As READ commands are issued sequentially to each BGMU in the channel, the VOC values are returned to the memory controller, which updates the running maximum whenever a new VOC is received (Algorithm 1, line 1–8). The controller then issues ACT commands to activate the target row in

TABLE II
LLM CONFIGURATIONS USED IN EXPERIMENTS.

Model	Layer	Head	KV_head	Emb_d	Seq_len	Inter_size
GPT-baby	6	6	6	384	256	1536
GPT-3-small	12	12	12	768	1024	3072
TinyLlama	22	32	4	2048	2048	5632
Llama3.2-3B	28	24	8	3072	4096	8192

TABLE III
SPARE-PIM CONFIGURATION

Component	Description	Component	Description
Technology	HBM3	Row / Column	8192 / 32
Die / Channel	1 / 4	Burst length	8
pCH	2	Clock freq.	1 GHz
Bankgroup	8	Bandwidth	1024 GB/s
Bank	4	Capacity	8 GB
Timing parameters			
$t_{BL} = 4, t_{RP} = 14, t_{RCD} = 14, t_{RAS} = 34, t_{RRD_L} = 6,$ $t_{CL} = 18, t_{CCD_S} = 1, t_{CCD_L} = 2, t_{FAW} = 30$			

all banks, followed by an all-bank long-data SPM command whose latency is configured according to the maximum VOC, and finally issues PRE after the sparse kernel completes. If the maximum VOC collected from all BGMUs is zero, indicating there is no valid work in any bank of the channel, the controller branches on this condition and skips the entire ACT–SPM–PRE sequence for that token, proceeding directly to the next command (Algorithm 1, line 9–21).

To support *SPARE-PIM*, the memory controller must incorporate additional logic that processes BGMU metadata and configures the SPM execution length. This enables the architecture to preserve all-bank execution while minimizing unnecessary latency and memory-access energy consumption.

VI. EVALUATION

A. Experimental Setup

Elemax. To evaluate the accuracy of Elemax, we use four models of different sizes, whose detailed configurations are summarized in Table II. We select datasets from a variety of tasks (*e.g.*, WikiText-103 [34], OPUS-100 [35], CNN/DailyMail [36], Python-Codes-25K [37], OpenMathInstruct-1 [38], and OpenWebText2 [39]) and split each dataset into training and validation sets. All models are trained from scratch in FP16 precision without pretrained weights. Llama3.2-3B is trained on two A100 80 GB GPUs, whereas the other models are trained on two RTX 3090 GPUs. Due to its larger scale, Llama3.2-3B is trained on the OpenWebText2 [39] dataset.

SPARE-PIM architecture. To evaluate *SPARE-PIM* architecture, we build an in-house simulator based on an open-source memory simulator [40]. *SPARE-PIM* architecture is designed on top of HBM3 [33], and its detailed configuration is summarized in Table III. For the CPU baseline, we use an Intel i9-10900X processor running at 3.7 GHz, with 10 cores and 19.25 MB of cache. For GPU experiments, we use an NVIDIA RTX 3090, which operates at 1.40 GHz (about 1.4× higher frequency than *SPARE-PIM* architecture) and provides 936.2 GB/s of memory bandwidth.